

```

# =====
# API WRAPPER
# =====

class OpenAIWrapper:
    """
    Wrapper for OpenAI API calls with error handling and retry logic.
    """

    def __init__(
        self,
        api_key,
        model_name=MODEL_NAME,
        max_retries=DEFAULT_MAX_RETRIES,
        retry_delay=DEFAULT_RETRY_DELAY,
    ):
        self.client = OpenAI(api_key=api_key)
        self.model_name = model_name
        self.max_retries = max_retries
        self.retry_delay = retry_delay

    def generate_vision_response(self, prompt, encoded_image):
        """
        Send prompt and encoded image to OpenAI API with retry logic.
        """
        last_error = None

        for attempt in range(1, self.max_retries + 1):
            try:
                logger.info(
                    f"Calling OpenAI API with model {self.model_name}, "
                    f"attempt {attempt} of {self.max_retries}"
                )

                response = self.client.responses.create(
                    model=self.model_name,
                    input=[
                        {
                            "role": "user",
                            "content": [
                                {
                                    "type": "input_text",
                                    "text": prompt,
                                },
                                {
                                    "type": "input_image",
                                    "image_url":
f"data:image/jpeg;base64,{encoded_image}",

```

```

        },
        ],
    }
    ],
    temperature=0.7,
)

logger.info("OpenAI API call completed successfully.")
return response.output_text

except (RateLimitError, APITimeoutError, APIConnectionError) as e:
    last_error = e
    wait_time = self.retry_delay * attempt

    logger.warning(
        f"Retryable API error on attempt {attempt}: "
        f"{type(e).__name__} - {e}"
    )

    print(
        f"WARNING in OpenAIWrapper.generate_vision_response(): "
        f"{type(e).__name__}\n"
        f"  Location: API attempt {attempt} of {self.max_retries}\n"
        f"  Message: {e}\n"
        f"  Action: Retrying after {wait_time} seconds."
    )

    time.sleep(wait_time)

except APIError as e:
    logger.error(f"OpenAI API error: {type(e).__name__} - {e}")

    print(
        f"ERROR in OpenAIWrapper.generate_vision_response(): APIError\n"
        f"  Location: OpenAI API request\n"
        f"  Message: {e}\n"
        f"  Suggestion: Check model name, request format, and OpenAI
account status."
    )
    raise

except Exception as e:
    logger.error(f"OpenAI API error: {type(e).__name__} - {e}")

    print(
        f"ERROR in OpenAIWrapper.generate_vision_response(): "
        f"{type(e).__name__}\n"
        f"  Location: OpenAI API request\n"

```

```
        f" Message: {e}\n"
        f" Suggestion: Check API key, model name, internet connection,
and request payload."
    )
    raise

    raise RuntimeError(
        f"OpenAI API request failed after {self.max_retries} attempts. "
        f"Last error: {last_error}"
    )

api_wrapper = OpenAIWrapper(api_key=api_key)
```